

1. 다음 평문을 아래 규칙으로 부호화한 최종 비트열을 쓰시오.

평문: PEPPER

규칙

- ㄱ. 부호화: 빈도가 가장 작은 두 노드를 합쳐 부모 노드를 만든다(반복).
- ㄴ. 빈도가 같으면 ASCII가 작은(빠른) 노드를 먼저 고른다.
- ㄷ. 두 노드를 합칠 때 ASCII가 작은 쪽을 왼쪽(0), 큰 쪽을 오른쪽(1)에 둔다.
- ㄹ. ASCII 순서: A(65) < B(66) < ... < Z(90).

3. 다음 코드의 출력 문자열을, 아래 규칙으로 부호화한 최종 비트열을 쓰시오.

```

1  def f(s):
2      cols = 2
3      rows = len(s) // cols
4      r = ""
5      for c in range(cols):
6          for row in range(rows):
7              r += s[row * cols + c]
8      return r
9
10 def g(s):
11     r = ""
12     for ch in s:
13         r += chr((ord(ch) - 65 + 1) % 26 +
14                 65)
15     return r
16 print(g(f("SUMMER")))

```

2. 다음 코드의 실행 결과를 쓰시오.

```

1  def f(s):
2      cols = 2
3      rows = len(s) // cols
4      r = ""
5      for c in range(cols):
6          for row in range(rows):
7              r += s[row * cols + c]
8      return r
9
10 def g(s):
11     r = ""
12     for ch in s:
13         r += chr((ord(ch) - 65 + 3) % 26 +
14                 65)
15     return r
16 print(g(f("GARDEN")))

```

규칙

- ㄱ. 부호화: 빈도가 가장 작은 두 노드를 합쳐 부모 노드를 만든다(반복).
- ㄴ. 빈도가 같으면 ASCII가 작은(빠른) 노드를 먼저 고른다.
- ㄷ. 두 노드를 합칠 때 ASCII가 작은 쪽을 왼쪽(0), 큰 쪽을 오른쪽(1)에 둔다.
- ㄹ. ASCII 순서: A(65) < B(66) < ... < Z(90).

4. 다음 코드의 출력 문자열을, 아래 규칙으로 부호화한 최종 비트열을 쓰시오.

```

1 s = "COFFEE"
2 key = [1, 0]
3 k = 2
4 r = ""
5 for b in range(0, len(s), k):
6     for j in key:
7         if b + j < len(s):
8             r += s[b + j]
9 print(r)

```

규칙

- ㄱ. 부호화: 빈도가 가장 작은 두 노드를 합쳐 부모 노드를 만든다(반복).
- ㄴ. 빈도가 같으면 ASCII가 작은(빠른) 노드를 먼저 고른다.
- ㄷ. 두 노드를 합칠 때 ASCII가 작은 쪽을 왼쪽(0), 큰 쪽을 오른쪽(1)에 둔다.
- ㄹ. ASCII 순서: A(65) < B(66) < ... < Z(90).

5. 다음 코드의 실행 결과를 쓰시오.

```

1 class Dog:
2     def __init__(self, name):
3         self.name = name
4     def bark(self):
5         print(self.name, "멍멍")
6
7 d = Dog("바둑이")
8 d.bark()

```

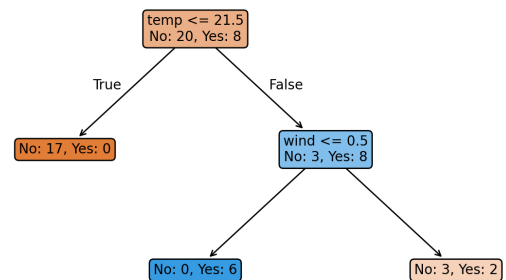
6. 다음 코드의 실행 결과가 2일 때, 빈칸에 들어갈 내용을 쓰시오.

```

1 class Counter:
2     def __init__(self):
3         self.cnt = 0
4     def add(____):
5         self.cnt = self.cnt + 1
6
7 c = Counter()
8 c.add()
9 c.add()
10 print(c.cnt)

```

7. 다음 의사결정트리를 보고 물음에 답하십시오. (temp(기온), wind(바람): 0=약함 1=강함)



입력 temp=28, wind=1 가 도달하는 칸(leaf)에 적힌 No 개수와 Yes 개수를 [No, Yes] 순서로 쓰시오.

10. 다음은 어떤 암 진단 모델의 혼동행렬이다. (Neg=암 아님, Pos=암. 행=실제, 열=예측)

actual	Actual: Neg	80	12
	Actual: Pos	13	15
		Pred: Neg	Pred: Pos
		predicted	

이 모델을 실제 암 진단에 사용할 때 어떤 문제가 있는지, 혼동행렬의 수치를 근거로 서술하시오.

12. 다음 코드의 출력 문자열을, 아래 규칙으로 부호화한 최종 비트열을 쓰시오.

```

1 s = "APPLE"
2 r = ""
3 for ch in s:
4     r += chr((ord(ch) - 65 + 1) % 26 + 65)
5 print(r)

```

규칙

- ㄱ. 부호화: 빈도가 가장 작은 두 노드를 합쳐 부모 노드를 만든다(반복).
- ㄴ. 빈도가 같으면 ASCII가 작은(빠른) 노드를 먼저 고른다.
- ㄷ. 두 노드를 합칠 때 ASCII가 작은 쪽을 왼쪽(0), 큰 쪽을 오른쪽(1)에 둔다.
- ㄹ. ASCII 순서: A(65) < B(66) < ... < Z(90).

11. 다음 코드의 실행 결과를 쓰시오.

```

1 s = "AAABBBCCD"
2 r = ""
3 i = 0
4 while i < len(s):
5     c = s[i]
6     cnt = 0
7     while i < len(s) and s[i] == c:
8         cnt += 1
9         i += 1
10    r += c + str(cnt)
11 print(r)

```

13. 다음 코드의 출력 문자열을, 아래 규칙으로 부호화한 최종 비트열을 쓰시오.

```

1 def f(s):
2     cols = 3
3     rows = len(s) // cols
4     r = ""
5     for c in range(cols):
6         for row in range(rows):
7             r += s[row * cols + c]
8     return r
9
10 def g(s):
11     r = ""
12     for ch in s:
13         r += chr((ord(ch) - 65 + 2) % 26 +
14 65)
15     return r
16 print(g(f("WINTER")))

```

규칙

- ㄱ. 부호화: 빈도가 가장 작은 두 노드를 합쳐 부모 노드를 만든다(반복).
- ㄴ. 빈도가 같으면 ASCII가 작은(빠른) 노드를 먼저 고른다.
- ㄷ. 두 노드를 합칠 때 ASCII가 작은 쪽을 왼쪽(0), 큰 쪽을 오른쪽(1)에 둔다.
- ㄹ. ASCII 순서: A(65) < B(66) < ... < Z(90).

14. 다음 코드의 실행 결과를 쓰시오.

```

1 def f(s):
2     cols = 2
3     rows = len(s) // cols
4     r = ""
5     for c in range(cols):
6         for row in range(rows):
7             r += s[row * cols + c]
8     return r
9
10 def g(s):
11     r = ""
12     for ch in s:
13         r += chr((ord(ch) - 65 + 1) % 26 +
14 65)
15     return r
16
17 def h(s):
18     r = ""
19     i = 0
20     while i < len(s):
21         c = s[i]
22         cnt = 0
23         while i < len(s) and s[i] == c:
24             cnt += 1
25             i += 1
26         r += c + str(cnt)
27     return r
28 print(h(g(f("AAAABBBB"))))

```

15. 다음 코드의 실행 결과를 쓰시오.

```

1 class Monster:
2     def __init__(self, hp):
3         self.hp = hp
4     def hit(self, dmg):
5         self.hp = self.hp - dmg
6
7 m = Monster(50)
8 m.hit(10)
9 m.hit(15)
10 print(m.hp)

```


이 코드가 무엇을 예측하는지 쓰고, 그 예측 결과가 분류와 회귀 중 어느 것에 해당하는지 이유와 함께 서술하시오.

[illegible]

1. 정답: 100110001**풀이**

부호화 대상: PEPPER

빈도: E:2, P:3, R:1

E(2) + R(1) -> ER(3): E=왼쪽(0), R=오른쪽(1)

ER(3) + P(3) -> ERP(6): ER=왼쪽(0), P=오른쪽(1)

코드: E=00, P=1, R=01

비트열: 1 00 1 1 00 01 = 100110001

정답: 100110001

3. 정답: 10010001101001**풀이**

코드 1단계: SUMMER -> SMEUMR

코드 2단계: SMEUMR -> TNFVNS

코드 출력: TNFVNS

빈도: F:1, N:2, S:1, T:1, V:1

F(1) + S(1) -> FS(2): F=왼쪽(0), S=오른쪽(1)

T(1) + V(1) -> TV(2): T=왼쪽(0), V=오른쪽(1)

FS(2) + N(2) -> FSN(4): FS=왼쪽(0), N=오른쪽(1)

FSN(4) + TV(2) -> FSNTV(6): FSN=왼쪽(0), TV=오른쪽(1)

코드: F=000, N=01, S=001, T=10, V=11

비트열: 10 01 000 11 01 001 = 10010001101001

정답: 10010001101001

2. 정답: JUHDGQ**풀이**

1단계: GARDEN -> GREADN

2단계: GREADN -> JUHDGQ

정답: JUHDGQ

4. 정답: 001000110101**풀이**

코드 1단계: COFFEE -> OCFEE

코드 출력: OCFEE

빈도: C:1, E:2, F:2, O:1

C(1) + O(1) -> CO(2): C=왼쪽(0), O=오른쪽(1)

CO(2) + E(2) -> COE(4): CO=왼쪽(0), E=오른쪽(1)

COE(4) + F(2) -> COEF(6): COE=왼쪽(0), F=오른쪽(1)

코드: C=000, E=01, F=1, O=001

비트열: 001 000 1 1 01 01 = 001000110101

정답: 001000110101

6. 정답: self**풀이**

메서드의 첫 매개변수는 self여야 self.cnt에 접근할 수 있다.

정답: self

7. 정답: [3, 2]**풀이**

입력 temp=28, wind=1 를 트리 맨 위에서부터 분기 조건(예/아니오)에 따라 내려간다.

도달한 칸: No 3, Yes 2.

정답: [3, 2]

5. 정답: 바둑이 멍멍**풀이**

d.bark()는 self.name(=바둑이)과 '멍멍'을 출력한다.

정답: 바둑이 멍멍

8. 정답: (1) k-NN(k-최근접 이웃). (2) 예: $k=3$ 또는 5. 짝수는 동점이 생기므로 홀수가 좋고, $k=1$ 은 이상치/노이즈에 민감, k 가 너무 크면 멀리 있는 점까지 포함되어 클래스 경계가 흐려진다. 데이터가 두 무리로 비교적 잘 나뉘므로 작은 홀수면 충분하다.

채점 포인트

- (1) KNeighborsClassifier는 k-NN(k-최근접 이웃) 분류기다.
- (2) 핵심 근거 3가지가 들어가면 정답 인정:
 - 홀수 권장(짝수는 동점).
 - $k=1$ 처럼 너무 작으면 노이즈/이상치 하나에 휘둘림.
 - k 가 너무 크면 먼 점까지 포함 -> 경계가 흐려지고 다수 클래스로 쏠림.
- 구체적 k 숫자(예: 3, 5)는 위 이유와 함께면 정답.

9. 정답: $B(R^2=0.94)$ 가 $A(R^2=0.34)$ 보다 성능이 좋다. R^2 가 1에 가까울수록 점들이 직선에 가깝게 모여 예측이 잘 맞기 때문이다. A는 점들이 직선에서 멀리 넓게 흩어져 있어(분산이 큼) 직선으로 예측하기 어려워 R^2 가 낮다.

채점 포인트

- R^2 (결정계수)는 1에 가까울수록 데이터가 직선에 잘 맞는다는 뜻.
- B의 $R^2(0.94)$ 가 A의 $R^2(0.34)$ 보다 크므로 B가 성능이 좋다.
- A가 낮은 이유: 점들이 직선에서 멀리 흩어져 있어(큰 분산) 직선이 잘 설명하지 못한다.
- (이상치/비선형 관계도 R^2 를 낮추는 원인으로 언급하면 가점)

10. 정답: 실제 암 환자(Pos) 28명 중 13명을 음성(Neg)으로 잘못 판정했다(위음성). 암 진단에서 위음성은 암 환자를 놓쳐 치료 시기를 놓치게 하는 치명적 오류다. 전체 정확도는 $(80+15)/120 \approx 79\%$ 로 나쁘지 않아 보이지만, 정작 암 환자의 약 46%(13/28)를 놓치므로 진단 모델로는 위험하다. 위음성을 줄이는 방향으로 개선해야 한다.

채점 포인트

혼동행렬 읽기: 실제 Pos(암) = 13(위음성) + 15(맞춤) = 28명.

위음성(FN)=13: 실제 암인데 음성으로 판정 -> 환자를 놓침.

암 진단에서는 위음성이 위양성보다 훨씬 위험하다(치료 시기 놓침).

정확도(약 79%)만 보면 괜찮아 보여도 암 환자를 절반 가까이 놓침 -> 부적합.

핵심: '위음성이 많아 암 환자를 놓친다'가 들어가면 정답.

12. 정답: 0001101001

풀이

코드 1단계: APPLE -> BQQMF

코드 출력: BQQMF

빈도: B:1, F:1, M:1, Q:2

B(1) + F(1) -> BF(2): B=왼쪽(0), F=오른쪽(1)

BF(2) + M(1) -> BFM(3): BF=왼쪽(0), M=오른쪽(1)

BFM(3) + Q(2) -> BFMQ(5): BFM=왼쪽(0), Q=오른쪽(1)

코드: B=000, F=001, M=01, Q=1

비트열: 000 1 1 01 001 = 0001101001

정답: 0001101001

11. 정답: A3B3C2D1

풀이

1단계: AAABBBCCD -> A3B3C2D1

정답: A3B3C2D1

13. 정답: 1110001000010011**풀이**

코드 1단계: WINTER -> WTIENR

코드 2단계: WTIENR -> YVKGPT

코드 출력: YVKGPT

빈도: G:1, K:1, P:1, T:1, V:1, Y:1

G(1) + K(1) -> GK(2): G=왼쪽(0), K=오른쪽(1)

P(1) + T(1) -> PT(2): P=왼쪽(0), T=오른쪽(1)

V(1) + Y(1) -> VY(2): V=왼쪽(0), Y=오른쪽(1)

GK(2) + PT(2) -> GKPT(4): GK=왼쪽(0), PT=오른쪽(1)

GKPT(4) + VY(2) -> GKPTVY(6): GKPT=왼쪽(0), VY=오른쪽(1)

코드: G=000, K=001, P=010, T=011, V=10, Y=11

비트열: 11 10 001 000 010 011 =
1110001000010011

정답: 1110001000010011

15. 정답: 25**풀이**

hp=50 -> hit(10) -> 40 -> hit(15) -> 25.

정답: 25

16. 정답: money 속성(잔액)에 입력값 x를 더한다(입금).**풀이**

deposit(x)는 self.money에 x를 더해 잔액을 늘린다.

정답: money 속성(잔액)에 입력값 x를 더한다(입금).

14. 정답: B2C2B2C2**풀이**

1단계: AAAABBBB -> AABBAABB

2단계: AABBAABB -> BBCCBBCC

3단계: BBCCBBCC -> B2C2B2C2

정답: B2C2B2C2

17. 정답: Pass

풀이

입력 study=8, attend=90 가 도달한 leaf: Fail 0개, Pass 5개.

더 많은 쪽으로 분류 -> Pass.

정답: Pass

19. 정답: 공부 시간(X)으로 시험 점수(y)를, 특히 7시간 공부했을 때의 예상 점수를 예측한다. 예측 결과는 숫자(회귀)다. 시험 점수처럼 연속적인 수치를 예측하므로 회귀이며, 범주(합격/불합격 등)를 고르는 분류와 다르다.

채점 포인트

fit(X, y)로 공부시간-점수 관계를 학습하고 predict(7)로 7시간일 때 점수를 예측.

점수는 연속적인 숫자 -> 회귀(LinearRegression).

범주를 고르는 분류와 구분되면 정답.

18. 정답: k=2: 가장 가까운 2개가 모두 class B이므로 B로 분류된다(동작은 정상이지만, 짝수 k는 동점이 생길 수 있어 일반적으로 권장되지 않는다). k=100: 전체 데이터가 20개뿐이라 이웃을 100개 볼 수 없고, 설령 전체를 본다고 해도 다수 클래스 A로만 분류되어 새 점의 위치(B에 가까움)가 무시된다. 따라서 둘 중 더 잘못된 설정은 k=100이다.

채점 포인트

새 점은 class B 무리에 확실히 가깝다.

k=2: 가장 가까운 2개가 모두 B -> B로 분류(위치는 반영). 다만 짝수 k라 동점 위험이 있어 이상적이진 않다.

k=100: 전체가 20개라 이웃 100개는 불가능(데이터 수 초과).

설령 전체를 이웃으로 보면 다수인 A로만 분류 -> 새 점 위치가 전혀 반영 안 됨.

결론: k=2는 사소한 결함(짝수)일 뿐이지만 k=100은 위치 정보를 통째로 잃는 근본적 오류 -> 더 잘못된 것은 k=100.

20. 정답: 1) 데이터 로드: csv 등에서 데이터를 불러온다. 2) 전처리: 결측치·이상치를 정리하는 등 데이터를 학습에 맞게 다듬는다. 3) 데이터 분리: train(학습용)과 test(평가용)로 나눈다. 4) 학습(fit): train 데이터로 모델을 학습시킨다. 5) 예측/평가: test 데이터로 예측하고 R2(회귀)·정확도(분류) 등으로 성능을 평가한다.

채점 포인트

데이터 로드 -> 전처리 -> (train/test 분리) -> 학습(fit) -> 예측/평가 순서.

전처리(결측치·이상치 정리)와 학습/평가 단계가 들어 가야 함.

순서가 맞고 각 단계가 무엇을 하는지 설명되면 정답.

